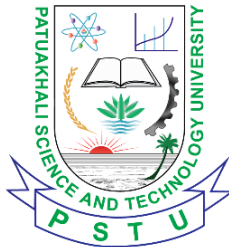


PATUAKHALI SCIENCE & TECHNOLOGY UNIVERSITY

FACULTY OF COMPUTER SCIENCE & ENGINEERING



A Report on TIC-TAC-TOE Game Using C++

Course Title: Computer Graphics and Image Processing

Course Code: CIT422

Submitted To

Md Mahbubur Rahman Assistant Professor Department of Computer Science and Information Technology Faculty of Computer Science and Engineering Patuakhali Science & Technology University	Farjana Sultana Mim Lecturer Department of Computer Science and Information Technology Faculty of Computer Science and Engineering Patuakhali Science & Technology University
---	---

Submitted By

Members Name	Members Id	Members Reg
Muhammad AL-Muzahid	1802034	08444
Hasan Morshed	1802023	08433
Hamid Mahtab Imad	1802039	08449
Saiful Islam	1802065	08475
Yeashen Arafath Rabib	1702001	07571

Submission Date – 27.09.2024

TIC-TAC-TOE Game Using C++

Introduction

This project aims to create a simple yet engaging Tic-Tac-Toe game using C++ for my Computer Graphics & Image Processing course. The primary objective was to implement graphical elements and user interaction, which are crucial aspects of computer graphics. The game features a graphical interface where the player can compete against a computer opponent in a 3x3 grid. Throughout the development process, we applied various concepts learned in the course, such as handling graphical rendering, managing user inputs, and creating basic AI logic.

Project Objectives

1. Develop a visually appealing and functional interface for the game using the graphics.h library.
2. Implement an AI opponent that can play against the player with basic decision-making capabilities.
3. Enable players to make moves using mouse clicks and provide feedback based on these inputs.
4. Create a structured game flow with win/lose conditions and an AI capable of making intelligent moves.

Development Tools

Language: C++

Graphics Library: BGI (Borland Graphics Interface)

Compiler: CodeBlocks with Graphics Support

Operating System: Windows

Features Implemented

- i. Main Screen and User Interface: The game starts with a welcome screen displaying the game title and a brief introduction. The interface is designed to be straightforward, with clear instructions and options for the player to choose their symbol ('X' or 'O').
- ii. Game Play: The game alternates turns between the player and the computer. The player makes moves by clicking on the desired cell, while the computer makes its move based on a simple AI algorithm.

- iii. AI Implementation: The computer opponent makes logical moves and can either block the player's winning move or secure a win for itself. The AI logic checks possible outcomes and decides on the best move accordingly.
- iv. Win/Lose Detection: The game continuously checks for a winning condition (three consecutive marks in a row, column, or diagonal) or a draw (no empty cells left).
- v. Restart and Exit Options: After the game ends, the player can choose to restart the game or exit. These options are made available through a graphical menu.

Code Overview

- i. Main Screen Setup (set_mainScreen1): This function initializes the graphical interface, sets up the title screen, and displays the game name with some visual effects to create a welcoming atmosphere.
- ii. User Input (choice_input1, choice_input2): These functions capture mouse clicks and determine whether the player wants to restart or exit the game. It ensures that the player's input is accurately registered and processed.
- iii. Game Logic (playerMove, AI): The playerMove function handles the player's move, while the AI function determines the computer's move using basic decision-making rules. The AI tries to either block the player from winning or take the winning move when available.
- iv. Winning Conditions (Winner): This function checks if there is a winning combination on the board. It is called after every move to determine if the game has ended in a win, loss, or draw.
- v. Graphical Elements (drawCross, drawCircle): These functions render the 'X' and 'O' symbols on the board. They are called whenever a move is made to update the board visually.

Challenges Faced

- i. Handling Mouse Inputs: It was challenging to detect and process mouse clicks within the game grid accurately. We had to fine-tune the coordinates and ensure precise click detection within each cell.
- ii. Implementing AI Logic: Balancing the AI to be neither too difficult nor too easy requires careful consideration of the decision-making rules. We initially faced issues with the AI permanently blocking the player, making the game less enjoyable.
- iii. Graphical Rendering: Another challenge was ensuring that the graphics were rendered correctly on different screen resolutions. We had to use dynamic scaling and relative positioning to handle this effectively.

Lessons Learned

This project gave us hands-on experience in applying computer graphics concepts to a practical application. We learned how to handle user inputs, manage graphical elements, and implement basic AI logic in a game environment. I also improved my skills in debugging and optimizing code for better performance and user experience.

Future Improvements

- i. Enhanced AI: The current AI logic is essential. We plan to incorporate more sophisticated strategies to make the game more challenging.
- ii. Multiplayer Mode: Adding a feature that allows two players to play on the same device would enhance the gameplay experience.
- iii. Improved Graphics: We would like to explore using more advanced graphics libraries to improve the visual quality and add animations.
- iv. Sound Effects: Adding sound effects would make the game more engaging.

Conclusion

This Tic-Tac-Toe helped us implement some of the crucial concepts from the Computer Graphics & Image Processing course. Watching the project perform as it should in real time was a very satisfying process. We are happy with the result and eager to hear your feedback and find ways to make it even better.